



Multiclass malware classification via first- and second-order texture statistics

Vinita Verma^{a,*}, Sunil K. Muttou^a, V.B. Singh^b

^a Department of Computer Science, University of Delhi, Delhi, India

^b Department of Computer Science, Delhi College of Arts and Commerce, University of Delhi, Delhi, India

ARTICLE INFO

Article history:

Received 10 September 2019

Revised 7 April 2020

Accepted 18 May 2020

Available online 23 July 2020

Keywords:

Code obfuscation

Malware visualization

Binary texture analysis

GLCM

Ensemble learning

Malware classification

ABSTRACT

The generally increasing volume of malware poses a challenge to the predominantly used static or dynamic analysis, which requires complex disassembly or time-intensive execution. Furthermore, high feature dimensionality and feature extraction costs per instance of malware increase overhead. The efficient classification of obfuscated malware, particularly for imbalanced classes, is a major challenge. This paper presents a visualization approach for malware classification to fill the gaps. Motivated by the visual similarity among malware from the same family, this paper proposes binary texture analysis over greyscale images created directly from their malware executables. The technique derives a novel combination of first-order and grey-level co-occurrence matrix (GLCM)-based second-order statistical texture features over the visualized malware. Using ensemble learning, the higher F1 score and accuracy obtained using Maling (a benchmark Windows malware dataset) relative to those of state-of-the-art techniques and Windows executables first submitted to VirusTotal from 2018 to 2019 indicate higher efficiency and reliability of the proposed technique for malware classification. Additionally, the proposed technique is robust to obfuscation methods (e.g., packing, code relocation, and encryption). Indeed, the technique uses relatively fewer features extracted without disassembly or code execution in less time, thereby significantly enhancing the scalability of classifying a large-scale malware corpus.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Malware, which is malicious in intent, is software that gains unauthorized access and damages computer systems. Malware includes viruses, worms, trojans, adware, spyware, or ransomware that disrupt the regular operation of the system, spy, and steal secret data, posing a threat to users and operating systems. Microsoft Windows is a popular desktop/laptop operating system. Forty-eight percent of malicious email attachments are Microsoft Office files, representing an increase of 5% from 2017 (Symantec Internet Security Threat Report, 2019). As stated in the Microsoft Security Intelligence Report (Volume 24 2018), malware encounter rates ranged from 5% to more than 7% in 2017, which have increased in subsequent years. It is crucial to detect and identify malware behaviour on Windows, which can be determined by identifying the family of the malware.

1.1. Malware analysis

One of the conventional detection methods matches the signature of the file under analysis with the signatures of known malicious programs. However, the method fails to detect unknown or new malware variants. Standard techniques include static and dynamic analyses. Static analysis examines malware without execution and disassembles the code (i.e., converts machine-level instructions to symbolic representation) to identify suspicious operations. Static analysis typically extracts opcodes (Ding et al., 2014) that determine the functionality of the code, function calls made by the executables (Hassen and Chan, 2017), or malicious sequences of API calls (Cheng et al., 2013; Gupta et al., 2016). Unlike static analysis, dynamic analysis executes the code in a controlled environment to monitor any malign actions. Dynamic analysis usually analyses system calls made by the malicious processes (Yuxin et al., 2011; Canali et al., 2012), malign file operations, and registry modification records (Cabau et al., 2016) or suspicious network activities (Nari and Ghorbani, 2013). Static analysis, together with dynamic analysis, has also been used for malware detection (Han et al., 2019).

* Corresponding author.

E-mail address: vinitaducs@gmail.com (V. Verma).

However, both types of analyses have their pros and cons. Static analysis faces a drawback due to code obfuscation. Software packing that compresses or encrypts a malicious file makes reverse engineering complex, and thus, static analysis requires an overwhelming effort. Static analysis requires unpacking, decompression or decryption, and disassembly before analysis, which are expensive and time-consuming. On the other hand, dynamic analysis executes the code, which is time- and resource-intensive. Executing malware also carries a risk of affecting the system. Moreover, malware execution often shows restrained behaviour in the running environment. Thus, both static and dynamic analyses lack speed, making either unscalable for large-scale malware analysis.

The reuse of code to create new malware variants preserves certain spatial, structural and statistical properties at the binary level. This characteristic is perceivable when malware executables from the same family, transformed to images, appear similar in layout and texture. Motivated by this visual analysis, this paper proposes binary texture analysis of greyscale images created directly from their executables. Other code structure-based static malware detection uses abstract syntax trees (AST) and bigram/trigram. AST, however, can bypass obfuscation (Rusak et al., 2018; Ndichu et al., 2019). On the other hand, the higher the n in n -gram analysis is, the higher the accuracy will be, but at the cost of large feature dimensionality (Jain and Meena, 2011; Liangboonprakong and Sornil, 2013; Ahmadi et al., 2016). N -gram analysis mainly focuses on predicting the next word, limiting its effectiveness (Ndichu et al., 2019). Zak et al. (2017) found that the n -gram method did not learn much beyond what is found in the PE header and import sections of the binary. Moreover, byte n -grams of executable sections of the binary have been found more effective than assembly n -grams, also indicating the effectiveness of binary analysis over static detection. Unlike static or dynamic analysis, the technique extracts texture features from the binaries without disassembly or execution. To the best of our knowledge, the lowest feature dimensionality attained in lesser time renders the proposed technique capable of the speed and scalability for large-scale malware classification. The rest of the paper is organized as follows: Section 2 discusses the related work. Section 3 describes the methodology for the proposed technique, and the classification results are analysed in Section 4. Section 5 presents the conclusion, and Section 6 mentions the threats to validity.

2. Related work

Visualization-based detection using static or dynamic features includes opcode-level-based images (Manavi and Hamzeh, 2017; Ni et al., 2018), system call-based behavioural graphs (Park et al., 2010), API call-based malicious behaviour visualization (Shaid and Maarof, 2014; Kim et al., 2019) and disassembled code-based breadth-first search (Kim and Youn, 2017). Treemaps and thread graphs (Trinius et al., 2009) visualize section-wise API calls and opcode sequences, respectively, to trace malign actions.

Malware binaries converted to images have been detected based on wavelet decomposition-based GIST features (Nataraj et al., 2011a), yielding 97.18% accuracy across the Maling dataset. However, this process used 320 high-dimensional features, 54 ms for extraction and 1.4 s for classification; moreover, these were less than the parameters for bigram analysis, which used 5 s for extraction and 56 s for classification. The method also employed unpacking prior to analysis. Entropy-graph-based classification (Han et al., 2015) also failed to classify packed samples. Another GIST descriptor-based detection has been used (Makandar and Patrot, 2015; Nataraj and Manjunath, 2016; Zhou et al., 2017; Nataraj and Manjunath (2016) obtained a 97.4% accuracy across Maling. Local binary patterns (LBPs) (Luo and Lo, 2017) outperformed GIST feature-based classification, and the

Table 1
Image height for several file sizes.

File size	Image height
<10 kB	32
10 kB - 30 kB	64
30 kB - 60 kB	128
60 kB - 100 kB	256
100 kB - 200 kB	384
200 kB - 500 kB	512
500 kB - 1000 kB	768
>1000 kB	1024

intensity and wavelet features (Kancherla and Mukkamala, 2013) also performed slightly better than computationally expensive Gabor-based features. Sparse representation-based malware detection (Nataraj et al., 2015) has been presented using random projections, and it obtained 92.83% accuracy but relied on increasing dimensionality to 512 features, which took 24 s while using orthogonal matching pursuit (OMP) to identify the samples. Texture and global colour features combined with local features have been used to enhance detection (Fu et al., 2018). However, that technique necessitated parsing the malware structure and decryption or decompression before analysis. Deep learning-based convolutional neural networks (CNNs) (Cui et al., 2018; Gibert et al., 2019) applied to Maling achieved 94.5% and 98.48% accuracy, with running times of 0.02 s and 0.001 s, respectively. Gibert et al. (2019), however, used a large image size that increased the computational time. Another CNN approach (Mourtaji et al., 2019) obtained 97.02% accuracy. A hybrid deep learning cost-sensitive model using a CNN and bi-directional gated recurrent units (BiGRU) (Venkatraman et al., 2019) obtained a 91.6% F1 measure when applied to Maling. Another CNN and long short-term memory (LSTM) network model (Akarsh et al., 2019) resulted in 95.5% accuracy. Selection methods (Naeem et al., 2019) have been used to reduce the dimensionality of malicious patterns, obtaining 98% accuracy. However, such models are slower. The selection speed of local features using a hard assignment for clusters was slow, using 512 features and 4.27 s for classification. A generative adversarial network (GAN) model achieved 90% accuracy relative to 85% accuracy using a variational autoencoder (VAE) model across Maling (Burks et al., 2019). Hashemi and Hamzeh (2019) used binary texture analysis, extracting micro-patterns from binary transformed images. A comparison of binary texture analysis with dynamic analysis follows in (Nataraj et al., 2011b). The former resulted in higher accuracy, with 4000 times faster classification. A survey that reviews visualization-based malware classification techniques appears in Wagner et al. (2015).

Unlike existing visualization techniques, the proposed technique uses a statistical approach for binary texture analysis. To the best of our knowledge, it extracts novel features for Windows malware classification: first-order and grey-level co-occurrence matrix (GLCM)-based second-order texture statistics. The key achievement of the proposed technique is the relatively low dimensionality of 35 features extracted in a shorter time of approximately 37 ms, resulting in an average classification time of 0.01 s with better classification efficiency.

3. Proposed method

The method proposed in this paper uses image-based visualization of malware for classifying variants into their respective families. The malware binary code is read into a vector of 8-bit unsigned integers and is then organized into a matrix that is visualized as a greyscale image. The width of the image is fixed, and the height varies based on the file size, as shown in Table 1. Due to the reuse of code to create new malware variants, executables

of the same family retain similarity insofar as they preserve certain spatial, structural and statistical properties at the binary level. This characteristic renders texture similarity among the visualized malware (greyscale images) created from the executables belonging to the same family. Moreover, visualizing ability captures minor changes while retaining the global structure. Indeed, this paper proposes a binary texture analysis of visualized malware executables for classification. The visualized malware or image is first preprocessed via histogram equalization to obtain a uniform distribution of grey levels and is then resized to 64×64 pixels to reduce computational overhead. Then, first-order statistics are computed followed by the creation of GLCMs from the image to derive second-order texture statistics. Finally, statistics for both orders are merged to produce a feature set for classification. Section 3.1 provides insight into the features extracted.

3.1. Feature extraction

Image texture is a function of the spatial arrangement of its grey levels. The proposed technique uses a statistical approach to analyse the binary texture of malware. The spatial distribution and spatial relationship between the grey levels of a binary transformed image have been captured via first- and second-order texture features, respectively. Grey levels refer to the malware bytes in this context. Sections 3.1.1 and 3.1.2 describe the first- and second-order features, respectively, that have been used for texture analysis in this paper.

3.1.1. First-order statistical texture features

The first-order texture features describing spatial distribution of the grey levels over an $M \times N$ visualized malware V , computed by the proposed method, are as follows:

- 1) Mean (μ_1): This describes the calculation of an average grey level of the elements in V .

$$\mu_1 = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N V(i, j)$$

- 2) Median: This is the centre value in the arranged grey-level distribution, given by $median(V)$, and is more robust to outliers. When the mean and median are the same, grey levels are more or less evenly distributed.
- 3) Standard Deviation (SD): This quantifies the deviation in grey levels from the mean of the distribution, measured in the same unit as that of the data.

$$SD = \sqrt{\frac{1}{M \times N - 1} \sum_{i=1}^M \sum_{j=1}^N (V(i, j) - \mu_1)^2}$$

- 4) Skewness: This is a measure of asymmetry in the grey-level distribution. A positive skew indicates a longer tail on the positive or right side of the peak of distribution than on the left. A negative skew implies a long tail on the negative side of the distribution. The value is 0 for a perfectly symmetrical distribution.

$$skewness = \frac{\frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N (V(i, j) - \mu_1)^3}{SD^3}$$

- 5) Kurtosis: This defines how heavily the tails of the distribution vary from that of a normal distribution. High kurtosis indicates the presence of heavy tails or outliers, whereas a low value indicates a light-tailed distribution or lack of outliers relative to a normal distribution. A perfectly normal distribution has a kurtosis of three using the formula below:

$$kurtosis = \frac{\frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N (V(i, j) - \mu_1)^4}{SD^4}$$

- 6) Minimum: The lowest grey level present in the distribution, given by $\min(V)$.
- 7) 10th percentile: This is the value given by $perc_{10}$, such that 10% of the grey levels are less than and 90% are more than $perc_{10}$ in the distribution. This measure is robust to outliers.

$$perc_{10} = prctile(V, 10)$$

- 8) Maximum: This provides the highest grey level within the distribution, given by $\max(V)$.
- 9) 90th percentile: The value given by $perc_{90}$ separates the lowest 90% and highest 10% of grey levels in the distribution and is also robust to outliers.

$$perc_{90} = prctile(V, 90)$$

- 10) Interquartile Range (IQR): This is the difference between the upper quartile (75th percentile) and the lower quartile (25th percentile). This measure describes variability across the central 50% of values in the distribution. This measure is robust to outliers.

$$IQR = perc_{75} - perc_{25}$$

- 11) Mean Absolute Deviation (MAD): This gives an average absolute distance between the grey levels and the mean of the distribution.

$$MAD = \frac{\sum_{i=1}^M \sum_{j=1}^N |V(i, j) - \mu_1|}{M \times N}$$

- 12) First-order entropy: This measures randomness in the grey-level distribution that consists of greyscale values ranging from 0 to 255.

$$entropy = - \sum_{i=0}^{255} q(i) \log q(i)$$

$$q(i) = \frac{\text{number of times grey level } i \text{ occurs within the image}}{M \times N}$$

- 13) First-order energy: The energy composed of the elements in the distribution is defined as follows:

$$energy = \sum_{i=0}^{255} q(i)^2$$

- 14) Coefficient of Variation (CV): This is a standard measure of dispersion within a distribution. Unitless, this measure is useful in comparing the degree of variation from one grey-level distribution to another regardless of the scales of measurement.

$$CV = \frac{SD}{\mu_1}$$

- 15) Range: This is a calculation of the difference between the highest and the lowest grey level, given by $\max(V) - \min(V)$. It indicates the interval size comprising the distribution.

- 16) Quartile Coefficient of Dispersion (QCD): This statistic is more robust to outliers and compares two or more grey-level distributions.

$$QCD = \frac{perc_{75} - perc_{25}}{perc_{75} + perc_{25}}$$

- 17) Variance: This measures the spread of grey levels within the distribution.

$$variance = \frac{1}{M \times N - 1} \sum_{i=1}^M \sum_{j=1}^N (V(i, j) - \mu_1)^2$$

- 18) Median Absolute Deviation (MedAD): This measure gives the centre value of the arranged absolute deviation of the elements from their median within the distribution. It is a robust measure of variability, being more resilient to outliers.

$$MedAD = median(|V(i, j) - median(V)|)$$

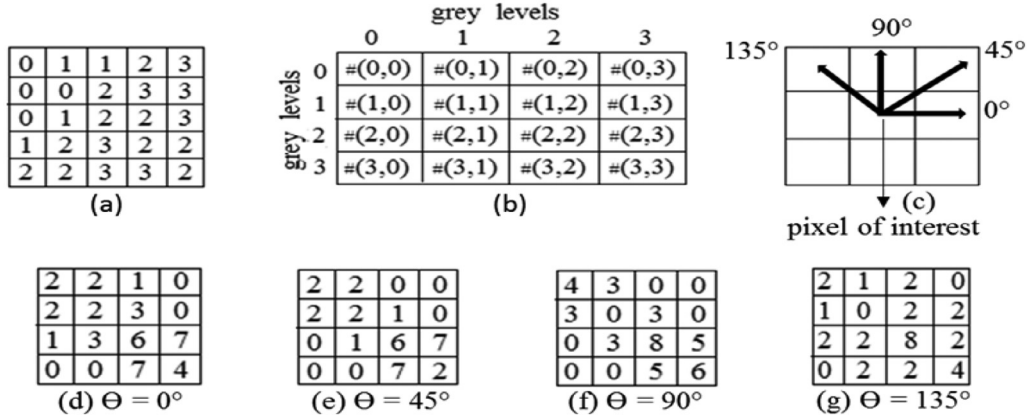


Fig. 1. (a) Image with 4 grey levels 0–3. (b) The general form of GLCM for an image with $L = 4$, with $\#(i, j)$ denoting the frequency of the grey levels with i and j being neighbours at distance d with angle Θ between them. (c) Rotational angle Θ between the grey levels. (d)–(g) GLCMs for $\Theta = 0^\circ, 45^\circ, 90^\circ$, and 135° , with $d = 1$ between the grey levels.

19) Root Mean Square: Also called the quadratic mean, this measure is calculated as follows:

$$RMS = \sqrt{\frac{\sum_{i=1}^M \sum_{j=1}^N V(i, j)^2}{M \times N}}$$

3.1.2. Second-order statistical texture features

Texture features for image classification (Haralick et al., 1973) are introduced under the assumption that texture information resides in an overall or average spatial relationship between the grey levels of an image. Specifically, this texture information is specified by a GLCM, also called a grey-level spatial dependence matrix. GLCM is computed and relies on three parameters (the number of grey levels L within an image, distance d and angular relationship θ between the grey levels). Therefore, GLCM is defined as an $L \times L$ matrix $G(i, j)$ of relative frequencies that specify the number of times the grey levels i and j separated by d in direction θ occur within the image. GLCM is computed as a symmetric matrix. Fig. 1(a) shows an image with $L = 4$, and Fig. 1(d–g) presents the GLCM matrices computed for this image using $d = 1$ and $\theta = 0^\circ, 45^\circ, 90^\circ$, and 135° between the grey levels, as shown in Fig. 1(c). Every 4×4 GLCM follows a template, as shown in Fig. 1(b). For example, for the GLCM in Fig. 1(d), the first element is calculated by counting the number of times the grey level 0 occurs horizontally ($\theta = 0^\circ$) with 0, with a distance of 1 between them. In Fig. 1(a), such an occurrence is found at position (1,0), (1,1) taken in this order, assuming row numbers begin at 0. A grey-level pair (0,0) also occurs at position in order (1,1), (1,0). Therefore, the frequency is 2. Similarly, a horizontal occurrence of grey-level pair (0,1) with $d = 1$ between the two levels happens at positions (0,0), (0,1) and (2,0), (2,1) in the image. Therefore, the frequency of the co-occurrence of levels (0,1) with $d = 1$ and $\theta = 0^\circ$ between them is 2. With GLCM being symmetric, the frequency of the co-occurrence of grey levels (i, j) is the same as that of (j, i) .

In this paper, the binary transformed greyscale image is reduced to 32 sufficient levels to reduce the computational cost for the matrix. Then, using the image, four GLCM matrices of 32×32 pixels each are computed, corresponding to $\theta = 0^\circ, 45^\circ, 90^\circ$, and 135° and $d = 1$ between the grey levels. The GLCMs obtained are divided by their respective sums to produce normalized matrices. Therefore, the normalized GLCM matrix $p(i, j)$ specifies the joint probability occurrence of the grey levels i and j within the image with a specific spatial relationship between them. A set of 8 texture features is derived from each of the four matrices p . Unlike first-order features that describe grey-level distribution us-

ing discrete values, second-order features use a joint probability distribution of grey levels to describe the spatial relationship between them. GLCM-based second-order texture features derived by the proposed method are as follows:

1) Contrast: This measures the amount of grey-level variation between the neighbouring pixels. High contrast corresponds to considerable variation in intensity across the image. Also known as inertia, the value 0 signifies a constant image.

$$contrast = \sum_{i=1}^L \sum_{j=1}^L |i - j|^2 p(i, j)$$

2) Angular Second Moment (ASM): Also called Energy, this measures uniformity in the image. The value 1 signifies a constant image.

$$ASM = \sum_{i=1}^L \sum_{j=1}^L p(i, j)^2$$

3) Normalized Inverse Difference Moment (normalized IDM): This is a measure of the homogeneity of the image. Because of the weighting factor $(1 + ((i - j)/L)^2)^{-1}$, contributions from inhomogeneous areas ($i \neq j$) are smaller. The result is a low value for inhomogeneous images and a relatively higher value for similar ones. Normalizing this feature value improves classification performance (Clausi, 2002).

$$normalized\ IDM = \sum_{i=1}^L \sum_{j=1}^L \frac{p(i, j)}{1 + ((i - j)/L)^2}$$

4) Joint Average (μ): This is the grey-level weighted sum of joint probabilities. The value calculated using either i or j gives the same result because GLCM is symmetric.

$$\mu = \sum_{i=1}^L \sum_{j=1}^L i p(i, j)$$

5) Sum of Squares or Joint Variance (σ^2): This measures the variance, assigning relatively higher weights to the elements that differ from the mean of the co-occurrence matrix.

$$\sigma^2 = \sum_{i=1}^L \sum_{j=1}^L (i - \mu)^2 p(i, j)$$

6) Joint Entropy: This measures the degree of disorder or randomness in the co-occurrence matrix. Entropy is highest when all elements of the co-occurrence matrix are equal. The value is measured as follows:

$$joint\ entropy = - \sum_{i=1}^L \sum_{j=1}^L p(i, j) \log p(i, j)$$

7) Joint Maximum: This is the probability corresponding to the most common grey-level co-occurrence in image.

$$joint\ maximum = \max(p)$$

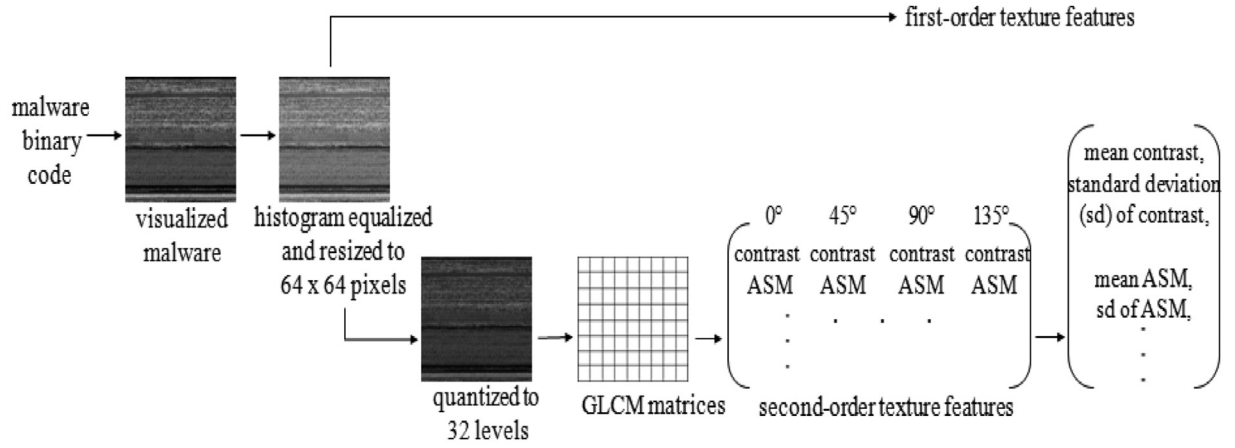


Fig. 2. Extraction of first- and second-order texture features from malware samples.

- 8) Correlation: This measures the linear dependence of grey levels on the neighbouring pixels.

$$\text{correlation} = \sum_{i=1}^L \sum_{j=1}^L \frac{(i - \mu)(j - \mu)}{\sigma^2} p(i, j)$$

Since these features are the function of distances and directions, we calculated standard deviations and means averaged over the four directions for each of the 8 features, yielding $2 \times 8 = 16$ second-order features per malware. Standard deviations and means, being translation and rotation invariant, make the proposed method robust to code relocation, which is one of the code obfuscation technique. Fig. 2 illustrates the proposed method.

Finally, sixteen second-order features are concatenated with nineteen first-order features, yielding 35 statistical texture features per malware. Additionally, the number of operations required to process an image to create the GLCM is proportional to the number of resolution cells present in the image. Every element in a GLCM is calculated using only two rows/columns of an image at a time, with no storage restrictions imposed.

4. Experiment and classification results

4.1. Dataset 1

The experimental data used in this paper are from a large-scale and imbalanced Windows malware dataset (Malimg, 2011) obtained from the Vision Research Lab, University of California, which is one of the benchmark datasets that has been used in many recent works (Nataraj et al., 2015; Nataraj and Manjunath, 2016; Cui et al., 2018; Akarsh et al., 2019; Burks et al., 2019; Gibert et al., 2019; Mourtaji et al., 2019; Naeem et al., 2019; Venkatraman et al., 2019). This dataset consists of 9339 malware samples from 25 families with 80–2949 samples per family, as shown in Table 2. Samples in the dataset are the executables that have been assigned labels provided by Microsoft Security Essentials. The families belong to the following categories: Worm, PWS, Dialer, Rogue, Backdoor, Trojan, and TDownloader. Fig. 3 shows examples of the greyscale images created directly from their executables. The figure shows texture similarity among malware from the same family and dissimilarity from those belonging to different families.

4.2. Classification results on dataset 1

After the extraction of texture features from the Malimg dataset, a low-dimensional feature set composed of 35 features per malware is obtained, thus reducing the complexity of the training

Table 2
Malimg dataset.

	Type	Family	Number of samples
1	Dialer	Adialer.C	122
2	Backdoor	Agent.FYI	116
3	Worm	Allaple.A	2949
4	Worm	Allaple.L	1591
5	Trojan	Alueron.gen!J	198
6	Worm:AutoIT	Autorun.K	106
7	Trojan	C2LOP.gen!g	200
8	Trojan	C2LOP.P	146
9	Dialer	Dialplatform.B	177
10	TDownloader	Dontovo.A	162
11	Rogue	Fakerean	381
12	Dialer	Instantaccess	431
13	PWS	Lolyda.AA1	213
14	PWS	Lolyda.AA2	184
15	PWS	Lolyda.AA3	123
16	PWS	Lolyda.AT	159
17	Trojan	Malex.gen!J	136
18	TDownloader	Obfuscator.AD	142
19	Backdoor	Rbot!gen	158
20	Trojan	Skintrim.N	80
21	TDownloader	Swizzor.gen!E	128
22	TDownloader	Swizzor.gen!I	132
23	Worm	VB.AT	408
24	TDownloader	Wintrim.BX	97
25	Worm	Yuner.A	800
			9339

model. The observations are divided into 10 equally sized folds using a stratified partition. This approach ensures that an approximately equal proportion of each family is present in each fold. Out of the 10 folds, one is used for testing, and the rest are reserved for training. We repeat this process 10 times, with each fold used exactly once for testing. In this 10-fold cross-validation, an ensemble-learning random forest classifier is used. Table 3 presents the obtained classification results in the form of a 25×25 confusion matrix C along with the precision, recall, and F1 measure for each family. These measures have been calculated using the parameters TP, FN, FP, and TN, which are usually defined for binary classification. For multiclass classification, these parameters for a class or a family 'i' are defined as follows: TP_i provides the number of correctly classified samples belonging to this family, given by $TP_i = C_{i,i}$. FN_i is defined as the number of misclassified samples that truly belong to family i and is calculated as $FN_i = (\sum_j C_{i,j}) - C_{i,i}$. FP_i gives the number of samples misclassified into this family, given by $FP_i = (\sum_k C_{k,i}) - C_{i,i}$, and $TN_i = (\sum_i \sum_j C_{i,j}) - TP_i - FN_i - FP_i$. Using these parameters, the precision for a family i is defined

Table 3

Confusion matrix along with the precision, recall and F1 measure for each family of the Malimg dataset.

	Adialer.C	Agent.FYI	Allapple.A	Allapple.L	Alueron.gen!J	Autorun.K	C2LOP.gen!g	C2LOP.P	Dialplatform.B	Dontovo.A	Fakerean	Instantaccess	Lolyda.AA1	Lolyda.AA2	Lolyda.AA3	Lolyda.AT	Malex.gen!J	Obfuscator.AD	Rbot!gen	Skintrim.N	Swizzor.gen!E	Swizzor.gen!I	VB.AT	Wintrim.BX	Yuner.A	Precision	Recall	F1 score
Adialer.C	122	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.98	1.0	0.99
Agent.FYI	0	116	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
Allapple.A	0	0	2910	38	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0.99	0.99	0.99
Allapple.L	0	0	29	1562	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.98	0.98	0.98
Alueron.gen!J	0	0	0	0	198	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
Autorun.K	0	0	0	0	0	106	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
C2LOP.gen!g	0	0	0	0	0	0	197	2	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0.99	0.99	0.99
C2LOP.P	0	0	0	0	0	0	3	139	0	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0.93	0.95	0.94
Dialplatform.B	0	0	0	0	0	0	0	0	177	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
Dontovo.A	0	0	0	0	0	0	0	0	0	162	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.99	1.0	1.0
Fakerean	0	0	0	0	0	0	0	0	0	0	379	0	0	0	0	0	2	0	0	0	0	0	0	0	0	1.0	0.99	1.0
Instantaccess	0	0	0	0	0	0	0	0	0	0	0	431	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
Lolyda.AA1	0	0	0	0	0	0	0	0	0	0	0	0	213	0	0	0	0	0	0	0	0	0	0	0	0	0.99	1.0	0.99
Lolyda.AA2	0	0	0	0	0	0	0	0	0	0	0	0	3	181	0	0	0	0	0	0	0	0	0	0	0	0.98	0.98	0.98
Lolyda.AA3	0	0	0	0	1	0	0	0	0	1	0	0	0	0	121	0	0	0	0	0	0	0	0	0	0	1.0	0.98	0.99
Lolyda.AT	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	158	0	0	0	0	0	0	0	0	0	1.0	0.99	1.0
Malex.gen!J	0	0	2	0	0	0	0	0	0	0	0	0	0	1	0	0	133	0	0	0	0	0	0	0	0	0.99	0.98	0.98
Obfuscator.AD	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	142	0	0	0	0	0	0	0	1.0	1.0	1.0
Rbot!gen	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	156	0	0	1	0	0	0	1.0	0.99	0.99
Skintrim.N	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	80	0	0	0	0	0	0.99	1.0	0.99
Swizzor.gen!E	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	112	14	0	0	0	0	0.88	0.88	0.88
Swizzor.gen!I	0	0	0	0	0	0	0	6	0	0	0	0	0	0	0	0	0	0	0	1	15	108	1	1	0	0.85	0.82	0.83
VB.AT	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	406	0	0	1.0	1.0	1.0
Wintrim.BX	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	97	0	0.99	1.0	0.99
Yuner.A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	800	0	1.0	1.0	1.0

as the number of correctly classified samples out of the total samples classified into this family. High precision relates to low false positives. Recall (sensitivity) gives the number of correctly classified samples out of the total samples belonging to this family. High recall implies low false negatives. The F1 measure, a more useful metric, considers both precision and recall. These measures are calculated as follows:

$$\text{precision}_i = \frac{TP_i}{TP_i + FP_i}$$

$$\text{recall}_i = \frac{TP_i}{TP_i + FN_i}$$

$$\text{F1 measure}_i = \frac{2 * \text{precision}_i * \text{recall}_i}{\text{precision}_i + \text{recall}_i}$$

Overall precision, recall, and the F1 measure are calculated using their respective measures averaged over individual families (macro-averages that give equal weight to each family), resulting in 98.04% precision, 98.06% recall, and a 98.05% F1 measure for the proposed technique. The accuracy (number of correct predictions out of the total predictions made) obtained with the proposed technique is 98.58%. These performance metrics indicate a higher classification efficiency of the proposed technique. A higher F1 measure implies both low false positives and false negatives, indicating that the proposed method achieves finer classification without being susceptible to large misclassifications.

A comparative analysis of the proposed technique relative to CNN-based techniques (Cui et al., 2018; Gibert et al., 2019; Venkatraman et al., 2019) across Malimg is presented in Table 4. In the study by Cui et al. (2018), an image of size 96×96 pixels was used for training, achieving a 94.5% F1 measure. Gibert et al. (2019) used a 128×128 pixel image, achieving a

94.8% F1 measure, which increased to 95.8% upon increasing the image size to 256×256 pixels. However, the proposed technique increased the F1 measure to 98.05% using a smaller 64×64 pixel image size. To the best of our knowledge, our technique has resulted in the highest classification accuracy (98.58%) compared with other techniques, as depicted in Fig. 4. The obtained accuracy is, however, only slightly higher than that of Gibert et al. (2019); the other performance metrics for the proposed technique are significantly higher, as shown in Table 4.

The dataset consists of both unpacked and packed samples. The ones that belong to the families Yuner.A, VB.AT, Malex.gen!J, Autorun.K, and Rbot!gen were packed with UPX packer. Different family samples packed with the same packer possess a similar structure and pattern, making it difficult to distinguish them. However, the proposed technique classified both Yuner.A and Autorun.K with 100% accuracy, Malex.gen!J with 98% accuracy, and VB.AT and Rbot!gen with 99% accuracy each. Additionally, Allapple has its code section encrypted in several layers using random keys. The proposed method classified Allapple.A and Allapple.L with 99% and 98% accuracy, respectively. Therefore, our technique is robust to both packing and encryption since it uses texture characteristics that are preserved by the weak encryption methods used by polymorphic engines. Moreover, the samples belonging to variants of the same family, such as Swizzor.gen!E and Swizzor.gen!I, are highly similar and difficult to differentiate. A CNN-based technique (Gibert et al., 2019) classified Swizzor.gen!E and Swizzor.gen!I with 71% and 62% accuracy, respectively, which were improved to 88% and 82% accuracy, respectively, using the proposed technique. The samples were efficiently classified into their respective families despite being the same type of malware, such as worms and trojans. As illustrated in Fig. 5, the textures of the executables of the Win32/Yuner.A worm and the Win32/AutoIT/Autorun.K worm are

Table 4
Comparison of the proposed technique with CNN-based techniques.

Classification measures	(Gibert et al., 2019)	(Cui et al., 2018)	(Venkatraman et al., 2019)	Proposed technique
Precision	95.8%	94.6%	91.8%	98.04%
Recall	96.56%	94.5%	91.5%	98.06%
Accuracy	98.48%	94.5%	96.3%	98.58%
F1 measure	95.8%	94.5%	91.6%	98.05%

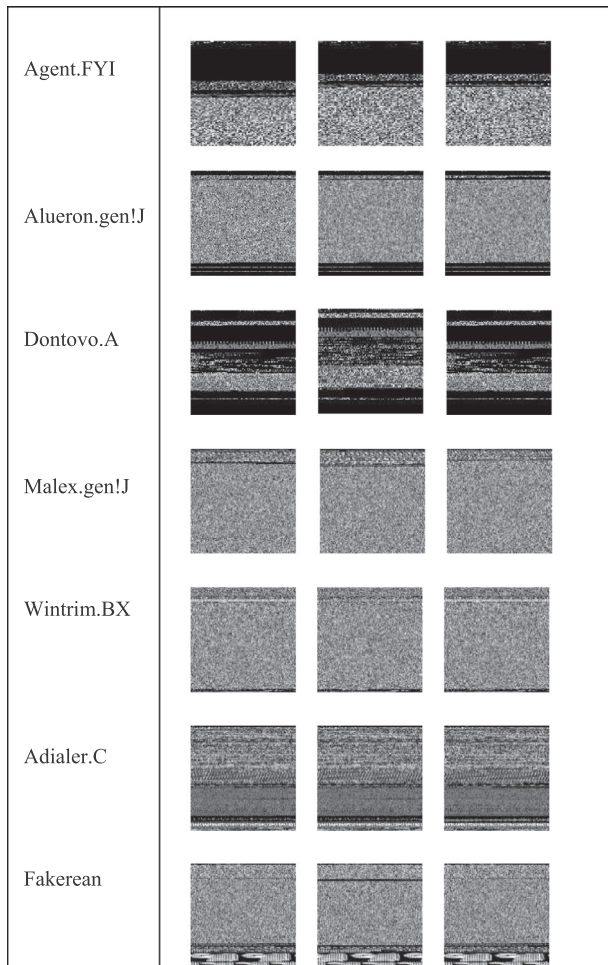


Fig. 3. Malware from the Maling dataset visualized as greyscale images. Note the texture similarity among malware from the same family.

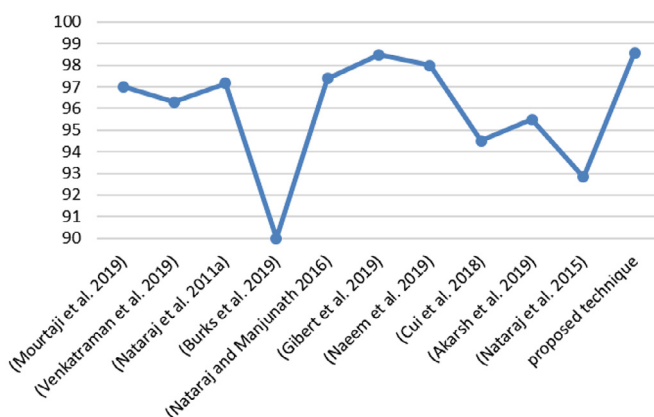


Fig. 4. Classification accuracy (%) obtained using different techniques across the Maling dataset.

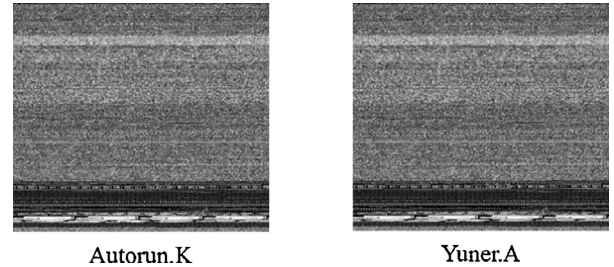


Fig. 5. High texture similarity between samples of the two families, classified with 100% accuracy each when using the proposed technique.

highly similar and indistinguishable to the human eye. However, the proposed technique classified the samples belonging to both Autorun.K and Yuner.A with 100% accuracy, an improvement over the 80% accuracy reported for Autorun.K (Gibert et al., 2019).

To evaluate the performance of the proposed technique on unknown malware samples (the ones whose families are not present in the training set), we trained the model in a leave-one-out manner by excluding an entire family from the training set and using it as a test set. The process was repeated 25 times, each time leaving out one family for testing and using the rest 24 for training. The behaviour is illustrated in Fig. 6, which shows the percentage of the test samples classified. Approximately 23% of the samples of Backdoor.Agent.FYI were classified into another Backdoor, Rbot!gen, and the other 33% and 41% of the samples of Agent.FYI were classified into PWSs that show similar behaviour of recording keystrokes, giving attackers access to the personal information and network resources. Similarly, 63% and 30% samples of PWS.Lolyda.AA2 were classified into Backdoor.Agent.FYI and another PWS, Lolyda.AA1, respectively. Eighty-eight percent of the samples of Trojan.Alueron.gen!J were classified into another Trojan, C2LOP.P. Similarly, Autorun.K, a worm, was classified into another worm, Yuner.A, and vice-versa with a classification rate of 1. A large percentage of Trojan.Skintrim was classified into another Trojan, C2LOP.P, and few samples were classified into Swizzor.gen!I and Wintrim.BX that show behaviour of downloading and executing arbitrary files similar to Skintrim. Similarly, TrojanDownloader.Swizzor.gen!I, was approximately 80% classified into the same family variant, Swizzor.gen!E, and was also classified into other TrojanDownloaders, Skintrim and Wintrim but with a minimal classification rate. Therefore, the proposed technique classified some unknown families into nearby families that show similar behaviour and into those belonging to the same malware category. However, relatively more families with large percentages were classified into dissimilar ones. Thus, the technique is not as effective for unknown malware classification as it is for known malware families.

Moreover, the proposed technique takes approximately 37 ms to extract all the first- and second-order texture features. This is comparably less time than the 54 ms needed to extract GIST features (a common approach) for texture analysis (Nataraj et al., 2011a; Nataraj and Manjunath, 2016). The proposed technique takes an average classification time of 0.01 s, which is less than that of other techniques and comparable to that of CNN-based techniques, as

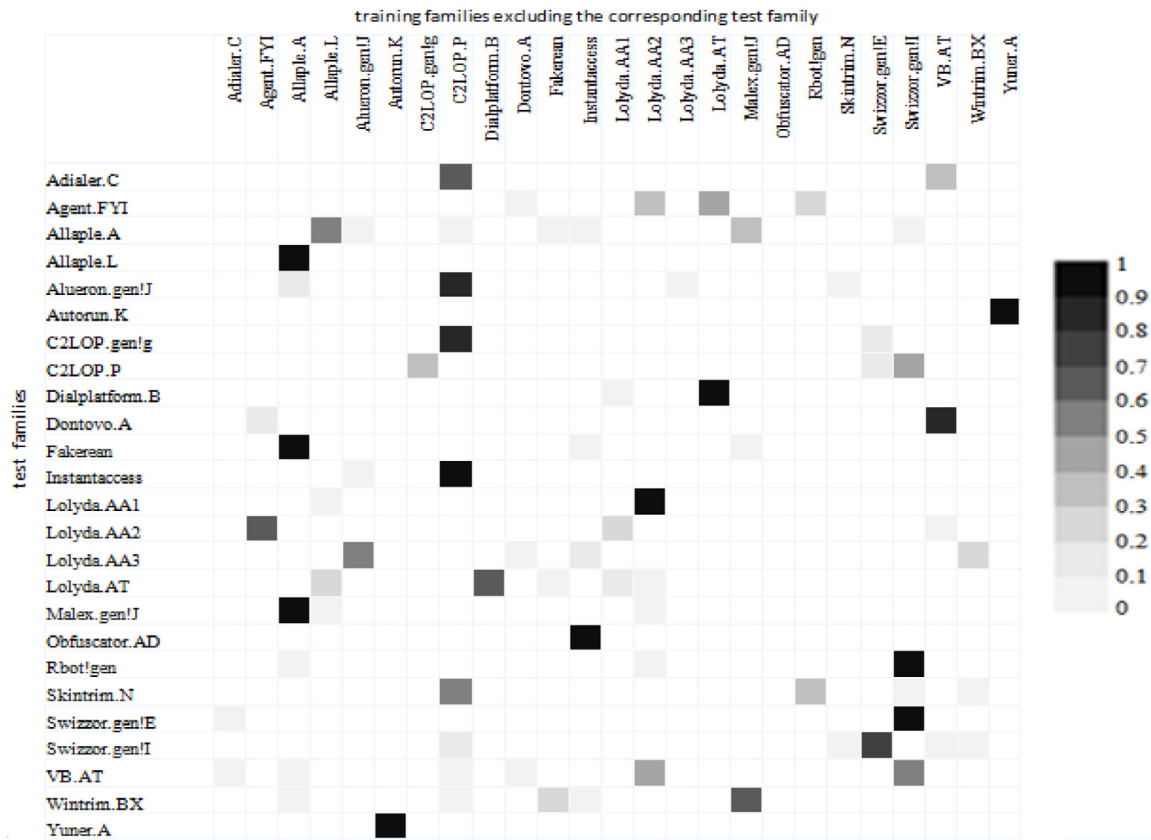


Fig. 6. Result of unknown malware classification.

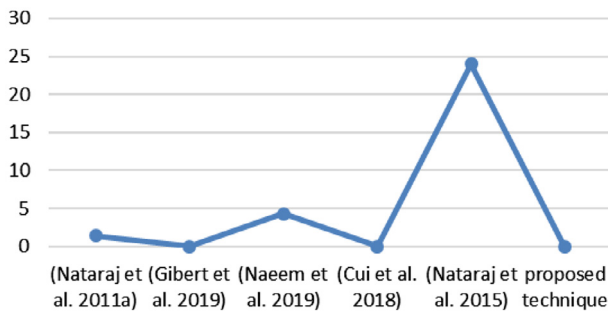


Fig. 7. Running time (sec) of different techniques.

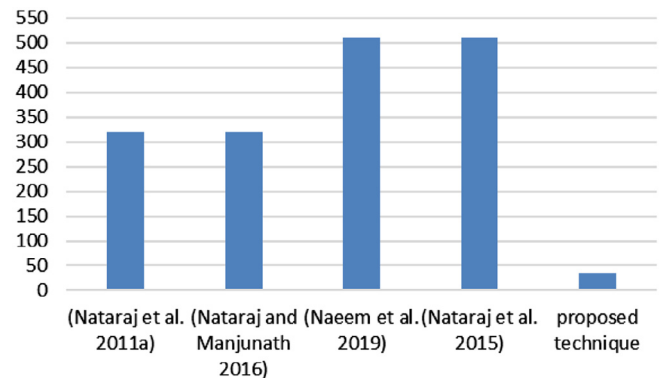


Fig. 8. Feature dimensionality of different techniques.

shown in Fig. 7. This short classification time can be attributed to the low-feature dimensionality obtained. The proposed technique uses 35 features for classification, which is the lowest relative to other techniques, as follows in Fig. 8.

4.3. Dataset 2

To validate the performance and reliability of the proposed technique, we collected approximately 2916 Windows malware samples from a malware repository (VirusShare, 2019). The VirusTotal (VT) report (VirusTotal, 2019) corresponding to a sample contains its 'first-seen', 'last-seen', 'first-seen-itw' and 'scanned-date'. 'First-seen-itw' represents the first time a sample appeared in the wild, and 'first-seen' indicates the first time this sample was submitted to VT for analysis. 'Last-seen' provides the last time the

sample was uploaded to VT, and 'scanned-date' is the date of current analysis. 'First-seen-itw' is the accurate date of the first appearance of the sample. However, we could not trace 'first-seen-itw' for all the samples since it was unavailable for some resources. Moreover, following the works that used 'first-seen' (Canzanese et al., 2013; Kantchelian et al., 2015; Carlin et al., 2017; Rhode et al., 2018; Rosenberg et al., 2018; Saleh et al., 2018), we also used 'first-seen' (i.e., submission date of the samples to VT) as the basis of malware collection. All the collected samples have 'first-seen' from 2018 to 2019, which ensures that these samples were submitted for the first time to VT for analysis during recent years (2018–19). A list describing the dataset can be accessed online (Dataset 2 2019). The list consists of MD5, SHA-1, and SHA-

Table 5

Dataset 2, consisting of malware samples submitted for the first time to VT during 2018–19.

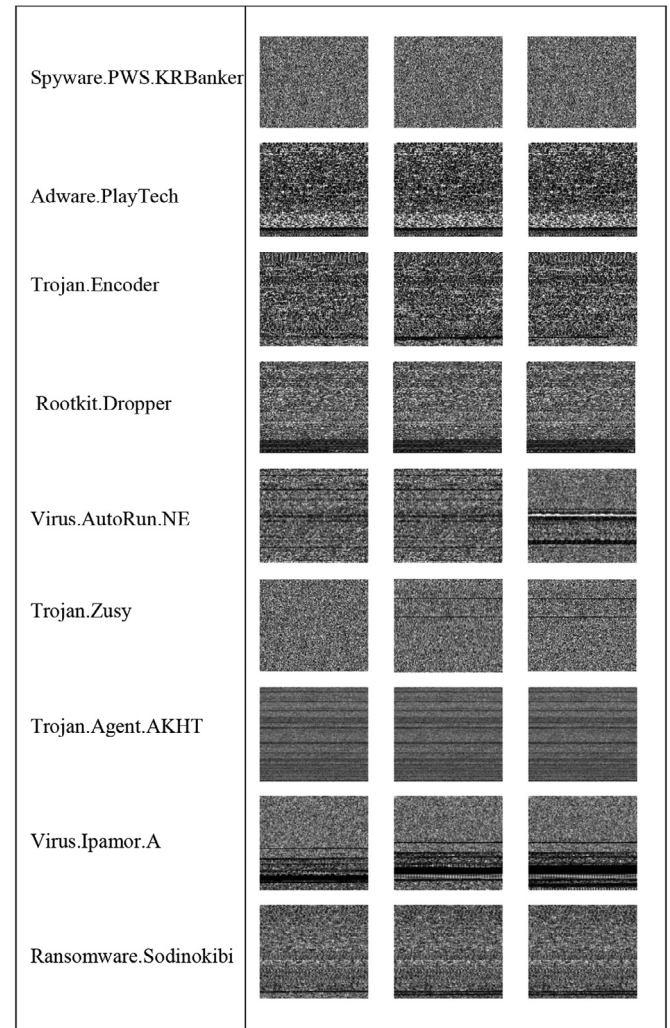
	Type	Family	No. of samples
1	Ransomware	WannaCry	73
2	Adware	PlayTech	284
3	Spyware.PWS	KRBanker	202
4	Trojan	AutoIt.D	252
5	Trojan	Encoder	206
6	Virus	AutoRun.NE	309
7	Trojan	Zusy	234
8	Rootkit	Dropper	21
9	Adware	Gator	124
10	Backdoor	Wabot.A	200
11	Trojan.PWS	Fareit	211
12	Trojan	Agent.AKHT	101
13	Spyware	Noon	224
14	Worm	Delf	31
15	TrojanDownloader	Banload	91
16	TrojanDownloader	Unrui.AA	109
17	TrojanSpy	Shiz	46
18	Virus	Ipamor.A	157
19	Ransomware	Sodinokibi	41
			2916

256 values and first-seen dates for the malware samples along with the VirusShare file names from which they were acquired. The samples are executables of 0.70 KB–7424 KB in size that were assigned the labels provided by the majority of the antivirus engines on VT, resulting in the dataset presented in Table 5. Therefore, Dataset 2 consisted of 2916 malware samples from 19 families with 21–309 samples per family. The samples belonged to the following categories: Ransomware, Spyware, Trojan, TrojanDownloader, Adware, PWS, Worm, Virus, Backdoor, and Rootkit. The executables were transformed into greyscale images using the procedure mentioned in Section 3 for texture analysis. Examples of visualized malware executables belonging to this dataset are provided in Fig. 9, which shows texture similarity among malware from the same family and dissimilarity among those belonging to different families.

4.4. Classification results for dataset 2

After extracting thirty-five texture features across Dataset 2, stratified partitions were used to divide the observations into 10 equal-sized folds. With 10-fold cross-validation, with every fold used precisely once for testing, a random forest classifier was used for classification. Table 6 presents the classification results in the form of a confusion matrix along with the precision, recall, and F1 measures for each family. The proposed technique achieved 97.44% precision, 98.12% recall, and a 97.75% F1 measure, thereby achieving finer classification efficiency for the created dataset.

However, Trojan:Win32/Encoder showed a 3.4% misclassification rate into Ransom:Win32/Sodinokibi. This phenomenon resulted from the fact that Trojan.Encoder behaves like a crypto-ransomware that encrypts the files on a system and demands a ransom to decrypt them. Ransom.Sodinokibi is also a crypto-ransomware. On the other hand, Ransom:Win32/WannaCry was classified with 100% accuracy despite not necessarily encrypting files and often acting as locker ransomware by locking the users out of their devices and demanding a ransom to unlock them. Trojan:Win32/Zusy and PWS:Win32/Fareit were misclassified into Spyware:Win32/Noon and vice-versa. Such misclassification can be attributed to their similar malicious intent of stealing login credentials and other sensitive data. However, Spyware.PWS.KRBanker and TrojanSpy.Shiz, both having spy capabilities, were each classified with 100% accuracy. The proposed technique achieving 98.11%

**Fig. 9.** Visualized malware belonging to Dataset 2.

accuracy over the recent malware samples indicates its reliability for malware classification.

We also evaluated the performance of the proposed texture analysis with other texture analyses that are outside the context of malware classification to provide more confidence in our approach. SVM-based distal lung image classification (Désir et al., 2012) and land-use scene classification (Chen et al., 2016) used local texture features (LBP) for classification. The LBP descriptor computes the distribution of binary patterns in the circular neighbourhood (characterized by radius R and number of neighbours P) of each pixel. The best-performing LBP feature vectors for a set of values ($R = 1, P = 8$), ($R = 2, P = 16$), and ($R = 3, P = 24$) followed by SVM classification (Désir et al., 2012) resulted in 94.14% and 94.03% accuracy for the Maling dataset and Dataset 2, respectively. Chen et al. (2016) used the multi-scale completed LBP (MS-CLBP) descriptor, providing two implementations. The MS-CLBP descriptor used kernel-based extreme machine learning for classification. The parameters that produced the best results were used in an experiment on both the datasets used in this paper. The CLBP1 descriptor, using $P = 12$ and $R = [1:8]$, yielded 96.08% and 94.07% accuracies for the Maling dataset and Dataset 2, respectively. CLBP2 using $R = 4$ and $P = 12$ using scales $[1:3]$ resulted in 94.25% and 92.01% accuracy for the Maling dataset and the other dataset, respectively. The results for the Maling dataset are included in Fig. 4 to illustrate the comparison with other methods

Table 6
Confusion matrix along with precision, recall, and the F1 measure for each family of Dataset 2.

	WannaCry	PlayTech	KRBanker	AutoIt.D	Encoder	AutoRun.NE	Zusy	Dropper	Gator	Wabot.A	Fareit	Agent.AKHT	Noon	Delf	Banload	Unruy.AA	Shiz	Ipamor.A	Sodinokibi	Precision	Recall	F1 Score
WannaCry	73	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
PlayTech	0	284	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
KRBanker	0	0	202	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
AutoIt.D	0	0	0	252	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.99	1.0	1.0
Encoder	0	0	0	0	194	0	2	0	0	0	3	0	0	0	0	0	0	0	7	0.97	0.94	0.95
AutoRun.NE	0	0	0	0	0	307	0	0	0	0	0	0	1	0	0	0	0	1	0	0.99	0.99	0.99
Zusy	0	0	0	0	2	0	223	1	0	0	0	0	5	0	1	1	0	0	1	0.98	0.95	0.97
Dropper	0	0	0	0	0	0	0	21	0	0	0	0	0	0	0	0	0	0	0	0.95	1.0	0.98
Gator	0	0	0	0	0	0	0	0	124	0	0	0	0	0	0	0	0	0	0	0.99	1.0	1.0
Wabot.A	0	0	0	0	0	0	0	0	0	200	0	0	0	0	0	0	0	0	0	1.0	1.0	1.0
Fareit	0	0	0	0	2	0	0	0	1	0	202	0	4	0	0	1	0	1	0	0.97	0.96	0.96
Agent.AKHT	0	0	0	0	0	0	0	0	0	0	0	101	0	0	0	0	0	0	0	1.0	1.0	1.0
Noon	0	0	0	1	0	2	2	0	0	1	3	0	213	0	2	0	0	0	0	0.95	0.95	0.95
Delf	0	0	0	0	1	0	0	0	0	0	0	0	0	30	0	0	0	0	0	0.97	0.97	0.97
Banload	0	0	0	1	1	0	1	0	0	0	0	0	2	1	85	0	0	0	0	0.97	0.93	0.95
Unruy.AA	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	109	0	0	0	0.98	1.0	0.99
Shiz	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	46	0	0	1.0	1.0	1.0
Ipamor.A	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	156	0	0.99	0.99	0.99
Sodinokibi	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	39	0.83	0.95	0.89

and the proposed method that obtained 98.58% and 98.11% accuracy for the Malimg dataset and Dataset 2, respectively.

5. Conclusion

This paper, in contrast to static or dynamic analysis, proposes a cost- and time-effective binary texture analysis for Windows malware classification. The proposed technique transforms executables into images and, to the best of our knowledge, has derived novel and lesser in number the texture statistics over malware samples in approximately 37 ms. The proposed technique has rendered an average classification time of 0.01 s, significantly enhancing the speed and scalability for classification. The higher F1 measures and accuracy of those proposed technique than those of state-of-the-art techniques and recent samples indicate the reliability and success of the proposed method. The technique also shows robustness towards code obfuscation and the class imbalance problem. The proposed digital image processing is an effective solution for malware classification that can complement existing detection methods as well.

6. Threats to validity

Despite finding some exciting features to represent code structure, the proposed technique has certain limitations. The approach can, however, be evaded by embedding malware into different larger legitimate binaries, changing the visual representation of the binary. Moreover, the technique is not very effective for detecting unknown malware (i.e., those whose families do not reside in the training set).

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

Vinita Verma: Conceptualization, Methodology, Software, Writing - original draft. **Sunil K. Mutttoo:** Validation, Resources, Writing - review & editing, Supervision. **V.B. Singh:** Validation, Data curation, Visualization, Supervision.

Acknowledgements

All the authors have substantially contributed to the work. The authors particularly thank VirusShare.com for providing access to their malware repository. Moreover, the research has not received any specific grant from any funding agencies in the public, commercial, or not-for-profit sectors.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.cose.2020.101895.

References

- Ahmadi, M., Ulyanov, D., Semenov, S., Trofimov, M., Giacinto, G., 2016. Novel feature extraction, selection and fusion for effective malware family classification. In: *Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy*. ACM, pp. 183–194.

- Akarsh, S., Simran, K., Poornachandran, P., Menon, V.K., Soman, P.K., 2019. Deep learning framework and visualization for malware classification. In: 2019 5th International Conference on Advanced Computing and Communication Systems (ICACCS). IEEE, pp. 1059–1063.
- Burks, R., Islam, K.A., Li, J., Lu, Y., 2019. Data augmentation with generative models for improved malware detection : a comparative study *. The IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference. IEEE.
- Cabau, G., Buhu, M., Oprisa, C.P., 2016. Malware classification based on dynamic behavior. In: 2016 18th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNAS). IEEE, pp. 315–318.
- Canali, D., Lanzi, A., Balzarotti, D., Kruegel, C., Christodorescu, M., Kirda, E., 2012. A quantitative study of accuracy in system call-based malware detection. In: Proceedings of the 2012 International Symposium on Software Testing and Analysis, ISSTA 2012. ACM, pp. 122–132.
- Canzanese, R., Kam, M., Mancoridis, S., 2013. Multi-channel change-point malware detection. In: Proceedings - 7th International Conference on Software Security and Reliability, SERE 2013. IEEE, pp. 70–79.
- Carlin, D., O' Kane, P., Sezer, S., 2017. Dynamic analysis of malware using run-time opcodes. In: Data Analytics and Decision Support for Cybersecurity, pp. 99–125.
- Chen, C., Zhang, B., Su, H., Li, W., Wang, L., 2016. Land-use scene classification using multi-scale completed local binary patterns. *Signal Image Video Process* 10 (4), 745–752.
- Cheng, J.Y.C., Tsai, T.S., Yang, C.S., 2013. An information retrieval approach for malware classification based on Windows API calls. In: 2013 International Conference on Machine Learning and Cybernetics. IEEE, pp. 1678–1683.
- Clausi, D.A., 2002. An analysis of co-occurrence texture statistics as a function of grey level quantization. *Can. J. Remote Sens.* 28 (1), 45–62. doi:10.5589/m02-004.
- Cui, Z., Xue, F., Cai, X., Cao, Y., Wang, G.G., Chen, J., 2018. Detection of malicious code variants based on deep learning. *IEEE Trans. Ind. Informatics* 14 (7), 3187–3196. doi:10.1109/TII.2018.2822680.
- Dataset 2. Windows malware dataset created. http://vinitaverma.site/malware_sample_files/list.html, 2019.
- Désir, C., Petitjean, C., Heutte, L., Thiberville, L., Salaün, M., 2012. An SVM-based distal lung image classification using texture descriptors. *Comput. Med. Imaging Graph* 36 (4), 264–270. doi:10.1016/j.compmedimag.2011.11.001.
- Ding, Y., Dai, W., Yan, S., Zhang, Y., 2014. Control flow-based opcode behavior analysis for Malware detection. *Comput. Secur.* 44, 65–74.
- Fu, J., Xue, J., Wang, Y., Liu, Z., Shan, C., 2018. Malware visualization for fine-grained classification. *IEEE Access* 6, 14510–14523. doi:10.1109/ACCESS.2018.2805301.
- Gibert, D., Mateu, C., Planes, J., Vicens, R., 2019. Using convolutional neural networks for classification of malware represented as images. *J. Comput. Virol. Hacking Tech.* 15 (1), 15–28. doi:10.1007/s11416-018-0323-0.
- Gupta, S., Sharma, H., Kaur, S., 2016. Malware characterization using Windows API Call sequences. In: Security, Privacy, and Applied Cryptography Engineering, SPACE 2016, pp. 271–280.
- Han, K.S., Lim, J.H., Kang, B., Im, E.G., 2015. Malware analysis using visualized images and entropy graphs. *Int. J. Inf. Secur.* 14 (1), 1–14. doi:10.1007/s10207-014-0242-0.
- Han, W., Xue, J., Wang, Y., Huang, L., Kong, Z., Mao, L., 2019. MalDAE: detecting and explaining malware based on correlation and fusion of static and dynamic characteristics. *Comput. Secur.* 83, 208–233. doi:10.1016/j.cose.2019.02.007.
- Haralick, R.M., Shanmugam, K., Dinstein, I., 1973. Textural features for image classification. *IEEE Trans. Syst. Man. Cybern.* (6) 610–621. doi:10.1109/TSMC.1973.4309314, SMC-3.
- Hashemi, H., Hamzeh, A., 2019. Visual malware detection using local malicious pattern. *J. Comput. Virol. Hacking Tech.* 15 (1), 1–14. doi:10.1007/s11416-018-0314-1.
- Hassen, M., Chan, P.K., 2017. Scalable function call graph-based malware classification. In: Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy. ACM, pp. 239–248.
- Jain, S., Meena, Y.K., 2011. Byte level n-gram analysis for malware detection. In: International Conference on Information Processing, pp. 51–59.
- Kancherla, K., Mukkamala, S., 2013. Image visualization based malware detection. In: 2013 IEEE Symposium on Computational Intelligence in Cyber Security (CICS), pp. 40–44.
- Kantchelian, A., Tschantz, M.C., Afroz, S., Miller, B., Shankar, V., Bachwani, R., et al., 2015. Better malware ground truth: techniques for weighting anti-virus vendor labels. In: AISEC 2015 - Proceedings of the 8th ACM Workshop on Artificial Intelligence and Security, co-located with CCS 2015. ACM, pp. 45–56.
- Kim, H., Kim, J., Kim, Y., Kim, I., Kim, K.J., Kim, H., 2019. Improvement of malware detection and classification using API call sequence alignment and visualization. *Cluster Comput.* 22, 921–929.
- Kim, J., Youn, J.M., 2017. Malware behavior analysis using binary code tracking. In: 2017 4th International Conference on Computer Applications and Information Processing Technology (CAIPT). IEEE, pp. 1–4.
- Liangboonprakong, C., Sornil, O., 2013. Classification of malware families based on n-grams sequential pattern features. In: 2013 IEEE 8th Conference on Industrial Electronics and Applications (ICIEA). IEEE, pp. 777–782.
- Luo, J.S., Lo, D.C.T., 2017. Binary malware image classification using machine learning with local binary pattern. In: 2017 IEEE International Conference on Big Data (Big Data), pp. 4664–4667.
- Makandar, A., Patrot, A., 2015. Malware analysis and classification using Artificial Neural Network. In: 2015 International Conference on Trends in Automation, Communications and Computing Technology (I-TACT-15). IEEE, pp. 1–6.
- Maling. Windows malware dataset from Vision Research Lab, University of California. <https://vision.ece.ucsb.edu/research/signal-processing-malware-analysis/>, 2011 (accessed 15.02.19).
- Manavi, F., Hamzeh, A., 2017. A new method for malware detection using opcode visualization. In: 2017 Artificial Intelligence and Signal Processing Conference (AISP). IEEE, pp. 96–102.
- Microsoft Security Intelligence Report Volume 24. <https://www.microsoft.com/en-us/security/operations/security-intelligence-report/>, 2018 (accessed 04.02.2019).
- Mourtaji, Y., Bouhorma, M., Alghazzawi, D., 2019. Intelligent framework for malware detection with convolutional neural network. In: Proceedings of the 2nd International Conference on Networking, Information Systems & Security. ACM.
- Naeem, H., Guo, B., Naeem, M.R., Ullah, F., Aldabbas, H., Javed, M.S., 2019. Identification of malicious code variants based on image visualization. *Comput. Electr. Eng.* 76, 225–237. doi:10.1016/j.compeleceng.2019.03.015.
- Nari, S., Ghorbani, A.A., 2013. Automated malware classification based on network behavior. In: 2013 International Conference on Computing, Networking and Communications, ICNC. IEEE, pp. 642–647.
- Nataraj, L., Karthikeyan, S., Jacob, G., Manjunath, B.S., 2011. Malware images: visualization and automatic classification. In: VizSec '11 Proceedings of the 8th International Symposium on Visualization for Cyber Security. ACM.
- Nataraj, L., Karthikeyan, S., Manjunath, B.S., 2015. SATVA: SpArsiTy inspired classification of malware Variants. In: Proceedings of the 3rd ACM Workshop on Information Hiding and Multimedia Security, pp. 135–140.
- Nataraj, L., Manjunath, B.S., 2016. SPAM: signal processing to analyze malware [Applications Corner]. *IEEE Signal Process. Mag.* 33 (2), 105–117. doi:10.1109/MSP.2015.2507185.
- Nataraj, L., Yegneswaran, V., Porras, P., Zhang, J., 2011. A comparative assessment of malware classification using binary texture analysis and dynamic analysis. In: Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence. ACM, pp. 21–30.
- Ni, S., Qian, Q., Zhang, R., 2018. Malware identification using visualization images and deep learning. *Comput. Secur.* 77, 871–885. doi:10.1016/j.cose.2018.04.005.
- Ndichu, S., Kim, S., Ozawa, S., Misu, T., Makishima, K., 2019. A machine learning approach to detection of JavaScript-based attacks using AST features and paragraph vectors. *Appl. Soft Comput.* J. 84. doi:10.1016/j.asoc.2019.105721.
- Park, Y., Reeves, D., Mulukutla, V., Sundaravel, B., 2010. Fast malware classification by automated behavioral graph matching. In: Proceedings of the Sixth Annual Workshop on Cyber Security and Information Intelligence Research. ACM.
- Rhode, M., Burnap, P., Jones, K., 2018. Early-stage malware prediction using recurrent neural networks. *Comput. Secur.* 77, 578–594. doi:10.1016/j.cose.2018.05.010.
- Rosenberg, I., Shabtai, A., Rokach, L., Elovici, Y., 2018. Generic black-box end-to-end attack against state of the art API call based malware classifier. In: International Symposium on Research in Attacks, Intrusions, and Defenses (RAID), pp. 490–510.
- Rusak, G., Al-Dujaili, A., O'Reilly, U.-M., 2018. AST-based deep learning for detecting malicious PowerShell. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. ACM, pp. 2276–2278.
- Saleh, M., Li, T., Xu, S., 2018. Multi-context features for detecting malicious programs. *J. Comput. Virol. Hacking Tech.* 14 (2), 181–193.
- Shaid, S.Z.M., Maarof, M.A., 2014. Malware behavior image for malware variant identification. In: 2014 International Symposium on Biometrics and Security Technologies (ISBAST). IEEE, pp. 238–243.
- Symantec Internet Security Threat Report. <https://www.symantec.com/security-center/threat-report/>, 2019 (accessed 04.02.2019).
- Trinius, P., Holz, T., Göbel, J., Freiling, F.C., 2009. Visual analysis of malware behavior using treemaps and thread graphs. In: 2009 6th International Workshop on Visualization for Cyber Security. IEEE, pp. 33–38.
- Venkatraman, S., Alazab, M., Vinayakumar, R., 2019. A hybrid deep learning image-based analysis for effective malware detection. *J. Inf. Secur. Appl.* 47, 377–389.
- VirusShare. Malware repository. <https://virusshare.com/>, 2019 (accessed 07.10.19).
- VirusTotal. Suspicious files analyzer. <https://www.virustotal.com/>, 2019 (accessed 16.10.19).
- Wagner, M., Fischer, F., Luh, R., Haberson, A., Rind, A., Keim, D.A., et al., 2015. A survey of visualization systems for malware analysis. *Eurographics Conference on Visualization (EuroVis)* (2015).
- Yuxin, D., Xuebing, Y., Di, Z., Li, D., Zhanchao, A., 2011. Feature representation and selection in malicious code detection methods based on static system calls. *Comput. Secur.* 30 (6–7), 514–524. doi:10.1016/j.cose.2011.05.007.
- Zak, R., Raff, E., Nicholas, C., 2017. What can n-grams learn for malware detection? In: 2017 12th International Conference on Malicious and Unwanted Software (MALWARE). IEEE, pp. 109–118.
- Zhou, X., Pang, J., Liang, G., 2017. Image classification for malware detection using extremely randomized trees. In: 2017 11th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID). IEEE, pp. 54–59.

Vinita Verma is a Ph.D. scholar (Information Security) in the Department of Computer Science, University of Delhi, India. She received her Master of Science degree in Computer Science and Bachelor of Science Honours degree in Computer Science

from the University of Delhi in 2017 and 2015 respectively. Her research interests include information security, malware detection, image processing, and data hiding.

Sunil K. Muttoo is a Professor in the Department of Computer Science, University of Delhi, India. He received his B.Sc. Gp 'A' in 1974, M.Sc. (Mathematics) in 1976, M.Phil. (Mathematics) in 1978, Ph.D. (Coding Theory) in 1982 from the University of Delhi and M.Tech (Computer Science and Data Processing) in 1990 from IIT Kharagpur, India. His research interests include Coding Theory, Cryptography, Steganography, and Digital Watermarking. He has more than ten years of teaching experience in colleges at Under Graduate Level and more than twenty-five years teaching experience in the Department of Computer Science at Post Graduate and Doctorate

Level. He is a life member of the Cryptographic Society of India and a member of ACM.

V.B. Singh is an Associate Professor in Delhi College of Arts & Commerce, University of Delhi. He received his B.Sc. degree in 1992 and M.C.A. degree from Engineering College, UP, India, in 1996. He received his Ph.D. degree in Software Reliability from the University of Delhi in 2008. He is a reviewer for more than five academic journals and Editorial Board member for two computer science journals. He is a member of the Big Data Initiative, Department of Science and Technology, Govt. Of India. His research interests include Mining Software Repositories, Empirical Software Engineering, and Software Reliability Engineering